

SIPU kernel代码开发手册



本文档主要介绍sikernel repo中，算子开发与提交的相关说明，具体的编程指导内容，可参见：

[📖 SIPU Programming Guide](#)

1 开发设备

通过在线excel表格协调资源，并更新使用状态，链接如下：

[📅 软件服务器分配](#)

可通过SSH直接登入服务器终端，用户名密码为个人办公通用的用户名密码

2 基础算子库

基础算子库代码仓库地址，链接如下：

<https://gitlabsoft.siorigin.com/oplib/sikernel>

3 开发准备

开发服务器已经为个人账户准备了开发环境，包括但不限于：

代码块

```
1 gcc/g++
2 cmake
3 python3
```

SIPU算子开发，额外需要安装jinja2 package，为了保持链路通畅，需要使用阿里源安装，如下：

代码块

```
1 pip install jinja2 -i https://mirrors.aliyun.com/pypi/simple/ --trusted-host mirrors.aliyun.com
```

然后，进入sikernel代码目录，source相关资源即可：

```
1 source setup.sh
```

可以通过blas代码进行编译运行的简单验证：

代码块

```
1 cd source/source_builtin/blas/L3/mma/tile_mma_32x32_f16_f16_f16_tiled_tensor
2 bash run.sh
```

有如下输出即证明编译执行通过：

```
[ArchModel] Device Create, sipu_id = 0, handle = 0x5653df224f40
[ArchModel] Device Connected, sipu_id = 0, handle = 0x5653df224f40
[10:44:48.268][SIRT][W][2946898] [memobj.cpp:56] Legacy compatibility: Converting device to 1U16C mode
[10:44:48.268][SIRT][W][2946898] [memobj.cpp:56] Legacy compatibility: Converting device to 1U16C mode
[10:44:48.268][SIRT][W][2946898] [memobj.cpp:56] Legacy compatibility: Converting device to 1U16C mode
kernel mma_f16_m32 completed successfully!
*****
*      *      *      *      *
*      *      *      *      *
*****  *****  *****  *****
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
[SIPU 0] threading on: false
[SIPU 0] thread_num: 1
[SIPU 0] step cnt: 171120
[SIPU 0] time elapsed: 4.01936s
[SIPU 0] simulation speed: 42573
[ArchModel] Device Destroy, sipu_id = 0, handle = 0x5653df224f40
```

4 算子开发流程

4.1 基本代码结构

按照目前sikernel算子库的构建规则，单个算子的代码结构如下图所示（以add为例）：

代码块

```
1 add/
2 |— build.sh           // 编译脚本
3 |— CMakeLists.txt     // CMake file
4 |— kernel
5 |   |— add_kernel.su  // 核函数代码
6 |— run.sh             // 跑测执行脚本
7 |— test
8   |— test_host.cpp    // 单元测试代码
```

算子开发通常需要构建2份源代码，包括：

(1) 核函数（kernel）与封装（launch）代码，用于实现SIPU kernel的具体逻辑，设定合适的启动参数（grid, cluster, block等），形成的API，即为外部调用的接口，一般为*.su文件

(2) 测试代码，服务于本kernel的单元测试代码，内容一般包含：测试数据的构建，CPU逻辑的实现，kernel API的调用，kernel执行结果与CPU执行结果的比较等等

通过跑测脚本，可以实现完整的代码构建与跑测，生成内容一般有（以add为例）：

代码块

```
1 libadd.so    // 封装库
2 test_host    // 测试case（依赖封装库）
```

需要注意的是，算子的接口声明，需要统一放在sikernel的头文件中，路径如下：

代码块

```
1 include/sikernel.h
```

头文件中，需要为每个新增API编写注释，可使用///*启动默认注释格式，并补充相关注释细节。*

需要为默认生成的注释参数，增加[in] [out]说明，可参考如下：

代码块

```
1 ///  
2 ///  
3 ///  
4 ///  
5 ///  
6 ///  
7 template<typename T>  
8 void bincount(void *d_in, void *d_out, T dim_size, T min_len);
```

4.2 其他更新内容

算子代码开发完成后，需要通过本地测试代码测试验证，验证通过后，需要更新list文件，增加代码目录：

代码块

```
1 release/build/release.list
2 testcase/regression.list
```

4.3 CI测试

代码一般使用单独分支提交推送，为主分支提交merge request后，需要先在merge request链接下评论“test CI”触发CI测试，测试通过后，再申请合入主分支。

4.4 文档更新

开发测试完成后，需要在组内文档更新接口相关内容，用于记录开发进展，以及作为后续开发的参考：

 [OPLib kernel list](#)

需要增加对新开发算子的介绍，并在文档尾部表格中，增加相关算子的信息。

最后更新算子开发状态列表：

 [kernel dev plan](#)

4.5 代码提交方式

为避免算子仓库分支过多，需要将repo fork到个人名下做修改提交，流程如下：

代码块

- 1 fork出来repo到自己名下
- 2 然后clone自己名下的repo到本地
- 3 在本地 `git remote add upstream`
<https://gitlabsoft.siorigin.com/oplib/sikernel.git>
- 4 然后需要开发新代码之前 先 `git fetch upstream`
- 5 再`git merge upstream/dev`
- 6 这样同步repo后再切出来自己想要的分支
- 7 完成开发push 到自己的repo里
- 8 然后从自己的repo create MR到主repo的dev分支

提交MR时，需要在MR的标题中注明MR的编号，如：

[feat][xxx][!101]xxxxx

5 PyTorch接入验证方法

在PyTorch层级，对算子进行测试验证，更易于：

- (1) 构建测试输入数据
- (2) 进行精度比较（vs CPU）
- (3) 灵活调整实现CPU端的计算逻辑

而依赖torch_sipu进行PyTorch级别的算子测试，可能需要安装较多依赖，故考虑基于CPP extension构建一套sikernel专有的测试框架。

5.1 环境配置

PyTorch测试依赖于Python环境，建议为开发环境安装conda工具，便于灵活切换，安装配置方法如下：

代码块

```
1  (1) 下载miniconda安装包：
2  wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O
   miniconda.sh
3  (2) 运行安装脚本，将miniconda安装到个人目录下：
4  bash miniconda.sh
5  (3) 创建conda环境
6  conda create -n torch_sikernel python=3.10 -y
7  (4) 激活conda环境
8  conda activate torch_sikernel
9  (5) 更新pip source，在个人目录下增加.pip目录，并编写配置文件.pip/pip.conf，内容如下：
10 [global]
11 index-url = https://pypi.tuna.tsinghua.edu.cn/simple

12 [install]

13 trusted-host = pypi.tuna.tsinghua.edu.cn
14 (6) 安装必要依赖，如torch等cpu版本
15 pip install torch torchvision torchaudio
16 可使用pip list查看安装包，如下：
17 torch                                2.7.0+cpu
18 torchaudio                           2.7.0+cpu
19 torchvision                           0.22.0+cpu
20 后续测试验证过程中，可根据需要随时更新安装相应的package
```

5.2 编写接入代码

目前所有sikernel接入PyTorch的代码，统一放到以下文件中：

torch_benchmark/src/ext.cpp

该文件已将sum算子作为sample，编写了相关接入方法，主要有：

(1) PYBIND11_MODULE中增加PyTorch的接口定义，如：

代码块

```
1 m.def("sum", &sum_sipu, "sum op");
```

第一部分“sum”是算子在PyTorch中调用的名称

第二部分“sum_sipu”是算子被C++封装中的接口名称

第三部分“sum op”是对该算子的介绍说明

(2) 增加代码封装调用

实现C++封装接口逻辑，可以参考各个算子的测试逻辑（如tilecore_builtin_host.cpp），区别在于输入输出数据不再由主函数实现，而是由外部传入。另外，由于输入参数为tensor类型，需要结构相关参数信息，主要内容如下：

代码块

```
1 torch::Tensor input;  
2 void *data = input.data_ptr(); // tensor的数据部分  
3 int dim0_size = input.size(0); // tensor的shape参数，根据实际维度进行获取  
4 int dim0_stride = input.stride(0); // tensor的stride参数，根据实际维度进行获取  
5 input.dtype(); // tensor的数据类型，如torch::kFloat32等
```

封装代码中，由于外部传入的为CPU数据，需要使用sipu的API进行数据拷贝。

5.3 编写测试代码

PyTorch代码较为清晰简洁，可参考如下：

代码块

```
1 import torch  
2 import torch_sikernel as sipu  
3  
4 input = torch.rand((M, N), dtype=torch.float32, device='cpu')  
5 output = torch.empty((M), dtype=torch.float32, device='cpu')  
6 output_golden = torch.sum(input, dim=-1)  
7 sipu.sum(input, output)
```

重点内容是构建输入输出tensor（根据实际需要），大部分sikernel都有对应的torch算子（CPU版本），可直接调用执行。sikernel部分需要import，并使用相应的算子名称（如sum）来调用执行。执行完成后，可使用PyTorch的逻辑进行精度验证，sum的测试代码中，已经实现了常用的几种精度验证方法，包括：

(1) 余弦相似度，cosine similarity

(2) allclose

(3) 不等元素的个数

可直接参考print_accuracy的方法来调用执行（调用时，需要将2个对比的tensor reshape为1维）

5.4 编写测试脚本

由于对sikernel的测试，底层依然依赖各个算子的具体实现，可参考sikernel各个算子的编译配置方法，整合至torch测试环境中，torch_benchmark已经实现了一个sample脚本run.sh，如下：

代码块

```
1  #!/bin/bash
2
3  set -e
4
5  # run a sum operator test in PyTorch
6  rm -f *.so
7  rm -f *.elf
8
9  cd $SIKERNEL_ROOT_DIR/source/source_builtin/attention/sum/
10 rm -rf ./build
11 bash build_kernel.sh
12 cmake -B build ./
13 cmake --build build
14 cp ./build/*.so $SIKERNEL_ROOT_DIR/torch_benchmark/
15
16 cd $SIKERNEL_ROOT_DIR/torch_benchmark/
17 pip uninstall -y torch_sikernel
18 python setup.py install
19 python test_sum.py
20
```

重点内容是，进入到对应的算子目录内，进行elf和so的编译，并将so拷贝至torch_benchmark目录下。然后执行torch_sikernel的编译与安装，最终执行Python测试代码，即可进行完整的测试工作。

5.5 注意事项

(1) setup.py文件中，包含了torch_sikernel的诸多配置信息，如果有新增测试内容，可根据需要进行增删。如头文件部分，源码部分，依赖库部分等：

代码块

```
1     ext_modules=[
2         CppExtension(
3             name='torch_sikernel',
4             sources=['./src/ext.cpp'],
5             include_dirs=[header_path],
6             extra_compile_args={'cxx': ['-std=c++20']},
7             library_dirs=['./'],
8             libraries=['sipu', 'siformat', 'sipurt', 'sum']
9         )
10     ],
```

如上所示，如果测试新的算子，libraries需要对应更新。如果只需要测试关注的某个算子，其余依赖也可删除（如sum），来加快编译执行速度。

(2) 由于整个测试流程中，包含sikernel的编译与执行，也需要首先source sikernel目录下的setup.sh文件。

(3) 测试脚本中，也可根据需要，对不需要测试的内容进行注释，避免增加编译执行耗时。